Herramientas de visualización

Implementación e Integración de Dashboard Interactivo de Ventas | Actividad 1

Fecha límite de entrega: 26 de octubre 2026

Estudiante: César Goavanni Gómez Camargo

Repositorio del proyecto: [https://github.com/cgomezc88-svg/device-sales-visualization.git]

Contenido

1. Resumen ejecutivo 2
2. Arquitectura del Proyecto, Gobernanza e Infraestructura HTML/DOM 2
Metodología de Desarrollo: Spec-Driven Development (SDD) e Interacción Humano-AI 2
Infraestructura DOM y Carga de Dependencias 2
3. Descomposición Forzada del Dashboard (Google Charts y D3.js) 3
Layer 1: El Contenedor y la Infraestructura DOM 3
Layer 2: El Lienzo Gráfico y Estructuración Espacial 3
Layer 3: Fuente de Datos, Normalización y Filtro IQR 3
Layer 4: Escalas y Mapeo Multidimensional 3
Layer 5: Motor de Renderizado, Transiciones e Interactividad 4
4. Integración del Dashboard y Sincronización Dinámica 4
5. Guía de Capturas de Pantalla y Evidencias Visuales 4
Captura 1: Vista General del Dashboard Integrado 4
Captura 2: Interactividad y Tooltip Contextual en D3.js 6
Captura 3: Actualización Simultánea con Simulación Math.random() 6
Captura 4: Notación y Modal Informativo de Outliers (IQR) 8
Captura 5: Capacidad responsiva 8
Conclusiones y Hallazgos 9

1. Resumen ejecutivo

El presente trabajo aborda el desarrollo e integración de un *dashboard* interactivo y adaptable alojado en un único archivo web (index.html). El objetivo principal es ofrecer un entorno dinámico para el análisis comercial de distintos productos (*Smartphone A*, *Smartphone B*, *Tablet X*) analizando variables clave como ventas, ingresos y precio. Para lograrlo, se combinó una gráfica de barras implementada con la **Google Chart Library** y una gráfica de burbujas avanzada desarrollada con **D3.js v7**. Todo el sistema opera consumiendo datos estructurados en formato JSON embebido directamente en el DOM, sin requerir empaquetadores ni servidor web local (file://), e incorpora un mecanismo de simulación dinámica mediante Math.random() que permite actualizar ambas gráficas de forma simultánea al presionar el botón “Actualizar”.

2. Arquitectura del Proyecto, Gobernanza e Infraestructura HTML/DOM

Con la intensión de obtener un diseño ligero y compatible con cualquier navegador web, la aplicación se estructuró como un sistema Client-Side Only en un archivo único (index.html), prescindiendo totalmente de empaquetadores complejos como Webpack o Vite y de entornos como Node.js.

Metodología de Desarrollo: Spec-Driven Development (SDD) e Interacción Humano-AI

El desarrollo del dashboard se gestionó bajo la metodología **Spec-Driven Development (SDD)**. El proceso se gobernó de forma iterativa mediante especificaciones técnicas estrictas:

1. **Gobernanza de IA (**AGENTS.md **y** docs/constitution.md**):** Protocolo innegociable que estableció restricciones de archivo único, uso de CDNs oficiales, nomenclatura de código en inglés (camelCase) e interfaz/comentarios en español.
2. **Especificación Funcional (**specs/001-d3-html-page/spec.md**):** Redacción de requisitos bajo notación EARS (*Ubicuidad*, *Evento*, *Estado*) detallando el comportamiento de las escalas, tooltips y el filtrado por Rango Intercuartílico (IQR).
3. **Planificación y Tareas (**plan.md **y** tasks.md**):** Desglose modular en tareas secuenciales que guiaron la construcción del DOM y la integración de las librerías gráficas.

Infraestructura DOM y Carga de Dependencias

Las dependencias externas se cargaron exclusivamente mediante CDN oficial:

- **Google Charts Loader:** https://www.gstatic.com/charts/loader.js
- **D3.js v7:** https://cdnjs.cloudflare.com/ajax/libs/d3/7.9.0/d3.min.js

El árbol del DOM se organizó de manera vertical y responsiva en contenedores dedicados para el panel de controles globales (#controls), la gráfica superior de Google Charts (#google-chart-container), la gráfica SVG de D3.js (#chart), la ventana modal de calidad de datos (#outlier-modal) y el *tooltip* contextual (#tooltip).

3. Descomposición Forzada del Dashboard (Google Charts y D3.js)

Aplicando la metodología de Descomposición Forzada del MIT, el sistema se desglosa en 5 capas lógicas independientes:

Layer 1: El Contenedor y la Infraestructura DOM

- **Google Charts:** Se aloja en el contenedor neutro en la sección superior.
- **D3.js v7:** Se ubica debajo del gráfico anterior dentro de mediante un elemento vectorial <svg> configurado con viewBox="0 0 900 550" y preserveAspectRatio="xMidYMid meet".

Layer 2: El Lienzo Gráfico y Estructuración Espacial

- **Google Charts:** La librería calcula internamente los márgenes y ejes del diagrama de columnas.
- **D3.js v7:** Implementa la convención explícita de márgenes de D3, estructurando grupos vectoriales <g> aislados (#x-axis-group, #y-axis-group, #bubbles-group, #legend-group) para evitar colisiones visuales.

Layer 3: Fuente de Datos, Normalización y Filtro IQR

1. **Datos Embebidos:** Los datos residen en la constante JSON interna initialDataset dentro de index.html, por lo que no depende de un servidor para desplegar la información en las gráficas.
2. **Normalización:** Los valores nulos o indefinidos se reemplazan automáticamente por 0.
3. **Filtro IQR de Outliers:** Se evalúa el Rango Intercuartílico ($[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$). Los registros atípicos detectados se omiten del gráfico de burbujas y se notifican mediante el modal #outlier-modal.

Layer 4: Escalas y Mapeo Multidimensional

- **Google Charts (Columnas):** Mapea el eje X categórico por *Mes* y el eje Y cuantitativo por *Unidades Vendidas*, asignando barras diferenciadas por producto.
- **D3.js v7 (Burbujas):**
    - **Eje X (Ventas):** d3.scaleLinear().
    - **Eje Y (Ingresos):** d3.scaleLinear().
    - **Radio (Precio):** Escalado dinámico d3.scaleSqrt() con tamaño mínimo de r = 8px.
    - **Color (Producto):** Asignación ordinal (d3.scaleOrdinal()) con paleta fija: Smartphone A (#0098CD / #2B78E4, es decir azul con transparencia al 60% y sin transparencia), Smartphone B (#DB8100 / #E65100, es decir naranja con transparencia al 60% y sin transparencia) y Tablet X (#00566C / #27AE60, es decir verde con transparencia al 60% y sin transparencia).

Layer 5: Motor de Renderizado, Transiciones e Interactividad

- **Google Charts:** Instancia un objeto google.visualization.ColumnChart redibujándose de manera fluida.
- **D3.js v7:** Implementa el patrón Data-Join (enter-update-exit) con animaciones de transición suave (.transition().duration(800)). Los eventos de cursor (mouseover, mousemove, mouseout) activan el tooltip contextual con formato numérico y de moneda, mostrando el detalle de la observación.

4. Integración del Dashboard y Sincronización Dinámica

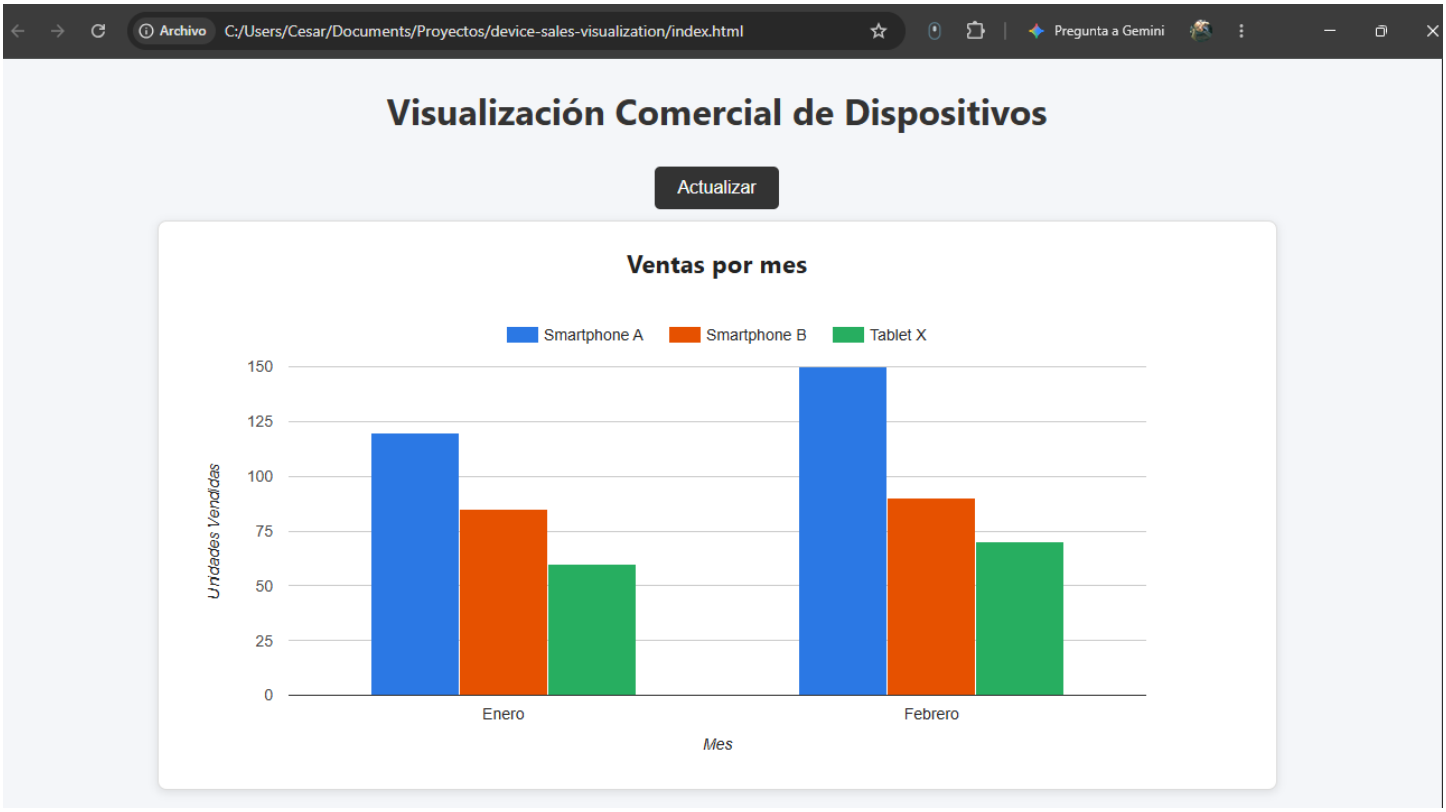
El dashboard unifica ambas visualizaciones bajo un flujo interactivo coherente:

- **Control Global de Actualización (**Math.random()**):** Al presionar el botón “Actualizar” (#update-btn), una función JavaScript simula métricas comerciales nuevas mediante variaciones aleatorias controladas, recomputando el dataset y redibujando simultáneamente el gráfico de columnas de Google Charts y el gráfico de burbujas de D3.js sin necesidad de recargar la página.

5. Guía de Capturas de Pantalla y Evidencias Visuales

Captura 1: Vista General del Dashboard Integrado

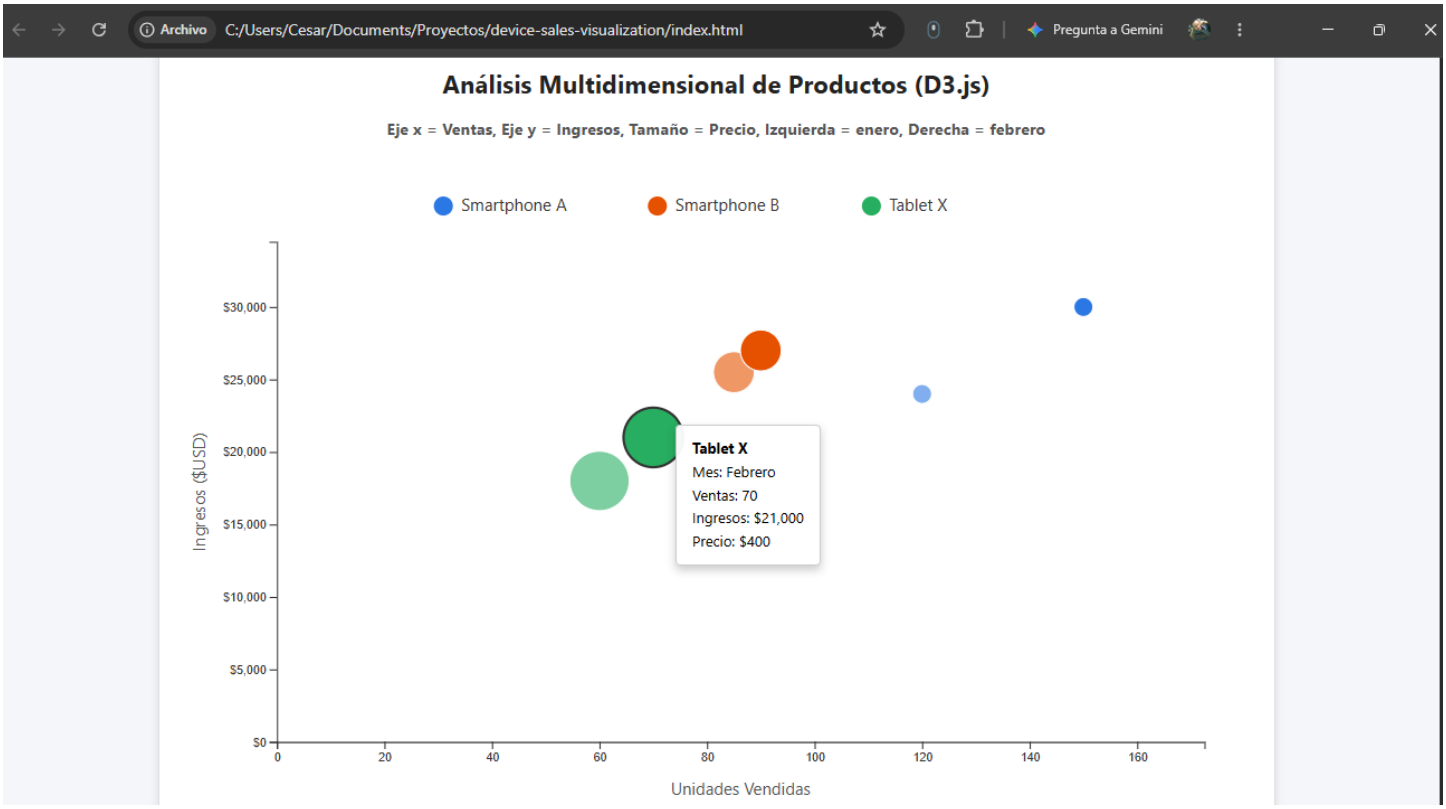
Captura de pantalla completa del dashboard en el navegador mostrando el diseño responsivo en una sola página. Se aprecia el encabezado, el panel de controles con el botón “Actualizar”, la gráfica de columnas agrupadas de Google Charts en la sección superior con su leyenda respectiva, y el gráfico multidimensional de burbujas de D3.js en la sección inferior con sus ejes formateados, la descripción requerida y la leyenda superior de colores por producto.


Archivo
C:/Users/Cesar/Documents/Proyectos/device-sales-visualization/index.html
Pregunta a Gemini
Visualización Comercial de Dispositivos
Actualizar
Ventas por mes
Smartphone A
Smartphone B
Tablet X
150
125
100
75
50
25
0
Unidades Vendidas
Enero
Febrero
Mes


Archivo
C:/Users/Cesar/Documents/Proyectos/device-sales-visualization/index.html
Pregunta a Gemini
Análisis Multidimensional de Productos (D3.js)
Eje x = Ventas, Eje y = Ingresos, Tamaño = Precio, Izquierda = enero, Derecha = febrero
Smartphone A
Smartphone B
Tablet X
$30,000
$25,000
$20,000
$15,000
$10,000
$5,000
$0
Ingresos ($USD)
0
20
40
60
80
100
120
140
160
Unidades Vendidas

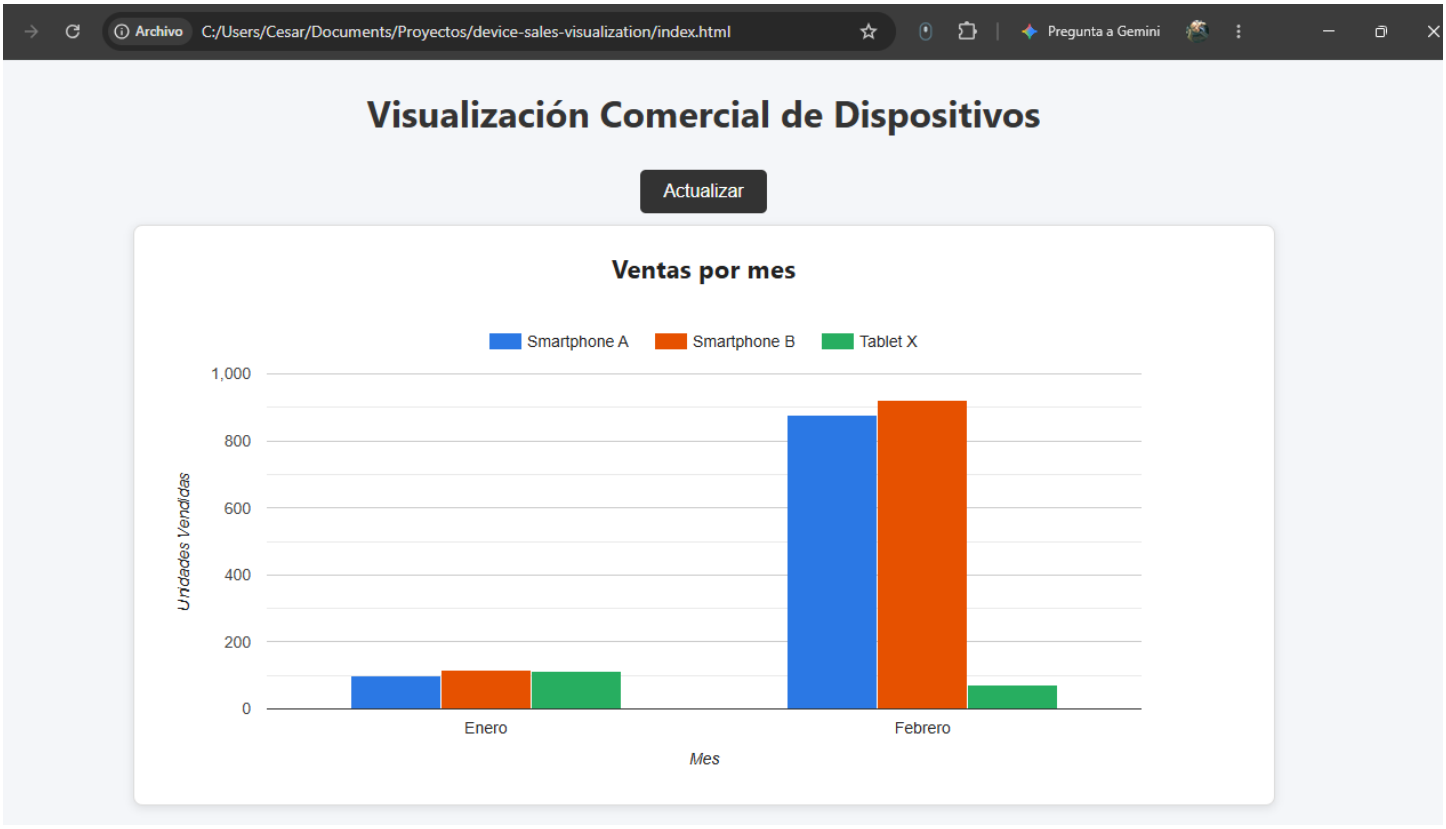
Captura 2: Interactividad y Tooltip Contextual en D3.js

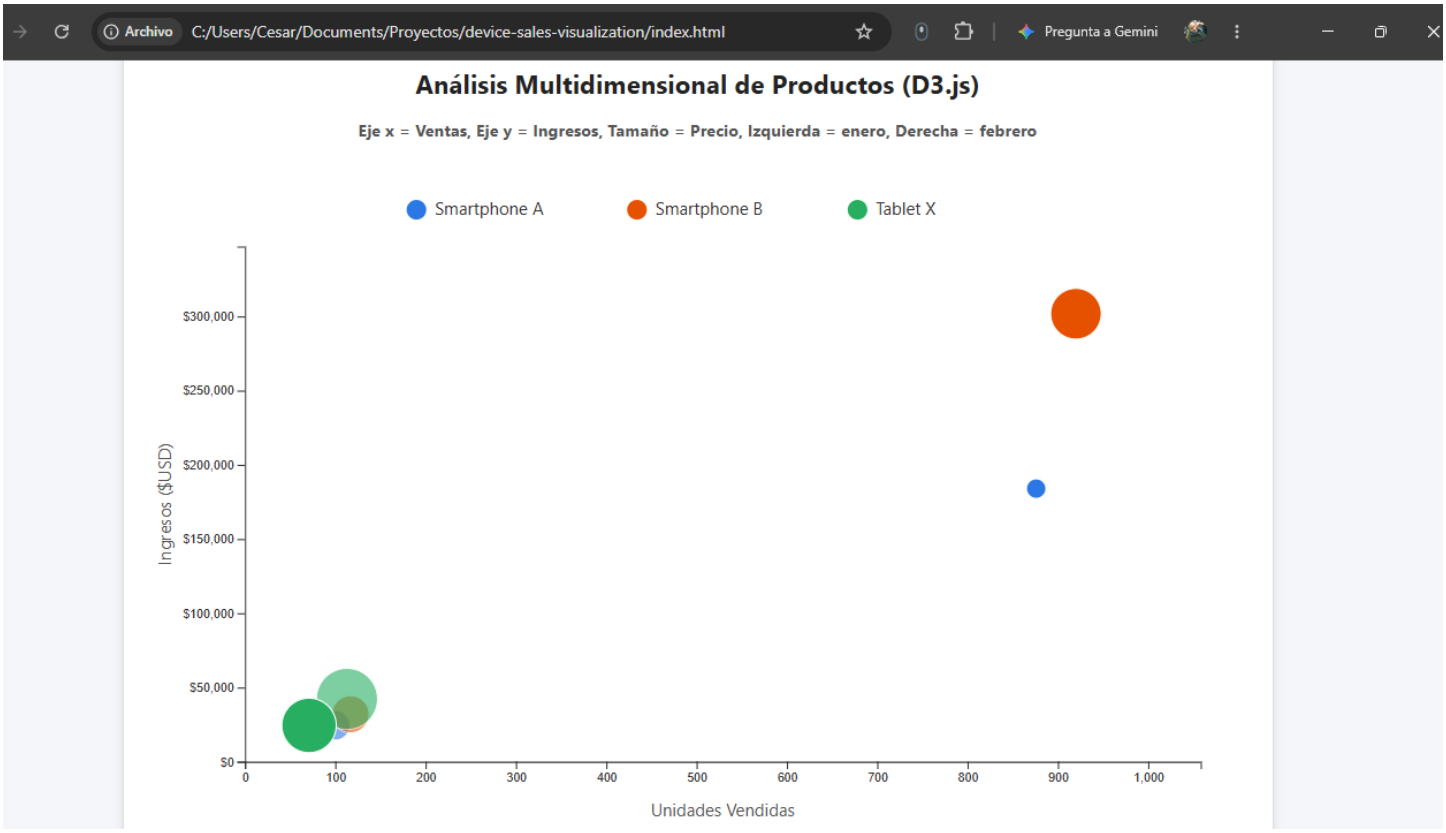
Se muestra el acercamiento al gráfico de burbujas de D3.js cuando el cursor se posiciona sobre un nodo (mouseover). Se observa la burbuja resaltada con un borde oscuro y la caja flotante del tooltip desplegada exactamente en la posición del puntero (pageX, pageY), mostrando claramente el nombre del producto, mes, ventas con formato de comas e ingresos/precio en formato de moneda ($USD).



Captura 3: Actualización Simultánea con Simulación Math.random()

Se hace evidente la secuencia visual al hacer clic en el botón “Actualizar”. Durante la ejecución, se aprecia la transición fluida donde las columnas de Google Charts y las burbujas de D3.js reajustan sus posiciones y tamaños de forma simultánea conforme a los nuevos datos simulados.


Archivo
C:/Users/Cesar/Documents/Proyectos/device-sales-visualization/index.html
Pregunta a Gemini
Visualización Comercial de Dispositivos
Actualizar
Ventas por mes
Smartphone A
Smartphone B
Tablet X
1,000
800
600
400
200
0
Unidades Vendidas
Enero
Febrero
Mes


Archivo
C:/Users/Cesar/Documents/Proyectos/device-sales-visualization/index.html
Pregunta a Gemini
Análisis Multidimensional de Productos (D3.js)
Eje x = Ventas, Eje y = Ingresos, Tamaño = Precio, Izquierda = enero, Derecha = febrero
Smartphone A
Smartphone B
Tablet X
$300,000
$250,000
$200,000
$150,000
$100,000
$50,000
$0
Ingresos ($USD)
0
100
200
300
400
500
600
700
800
900
1,000
Unidades Vendidas

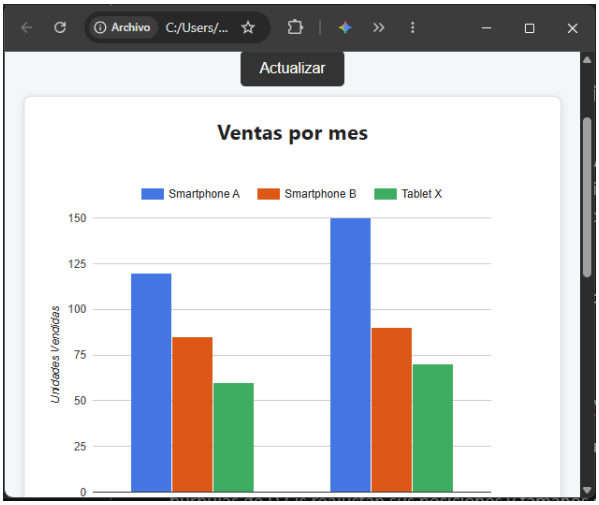
Captura 4: Notación y Modal Informativo de Outliers (IQR)

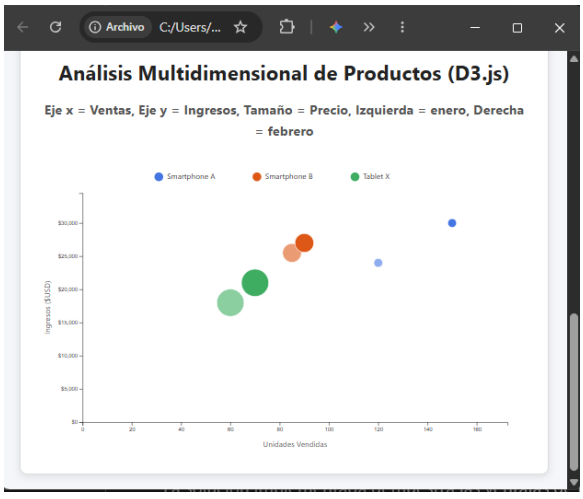
Captura de pantalla de la interfaz cuando se detecta un valor atípico en el dataset simulado. Se visualiza la ventana modal emergente (#outlier-modal) centrada en pantalla con fondo oscurecido (*backdrop*), mostrando el mensaje descriptivo sobre la omisión de registros atípicos calculados mediante el Rango Intercuartílico y el botón operativo para cerrarla.



Captura 5: Capacidad responsiva

Las visualizaciones se adaptan al tamaño de la página donde se despliegan.





Conclusiones y Hallazgos

La solución implementada demuestra las ventajas de una arquitectura basada en archivo único y datos embebidos. La combinación de Google Charts para comparativas categóricas directas y D3.js para análisis relacional multidimensional dota al usuario de una herramienta ejecutiva capaz de ser utilizada en entornos de negocio ejecutivos para la toma de decisiones en el caso de que se le proporcionara al dashboard la capacidad de conectarse a datos en tiempo real dentro de los volúmenes manejables de registros tanto para Google Chart como para D3.js. Asimismo, el uso de especificaciones estrictas bajo SDD, el filtrado estadístico IQR y las animaciones fluidas garantizan un producto de alta calidad técnica y visual, manteniendo el requerimiento actual en los entornos laborales profesionales “human in the loop” para garantizar un mayor nivel de determinismo al evitar el vibe coding.

El uso de <style> permitió mejorar la presentación de las gráficas mediante colores, posición de etiquetas, tipo de fuente, botones, entre otras características que permiten una interacción amigable con el dashboard.

El uso de <script src> permitió hacer uso de las herramientas Google Charts y D3.js, indispensables para la visualización de la gráfica de barras y de burbujas correspondientemente, así como de su configuración para un despliegue fluido y atractivo.

El uso de <main> donde se colocaron las visualizaciones de las gráficas.

El uso de <script> para implementar la lógica de las secciones documentadas en este reporte.

Finalmente se invita al lector a consultar el repositorio mostrado en la portada del presente reporte para profundizar en la resolución de esta actividad.